



MIAGE- L3

Systèmes Informatiques

Système d'exploitation

Les systèmes de fichiers

BAKAYOKO Ibrahima

Enseignant-chercheur en informatique

UFR Mathématiques & Informatique

Bakayoko.ibrahima1@ufhb.edu.ci

Open Our Mind for a free world

Imaginez un instant que votre système d'exploitation soit une bibliothèque immense, avec des milliers de livres (vos fichiers) soigneusement rangés sur les étagères. Le système de fichiers est le système de classification et d'indexation qui vous permet de retrouver rapidement le livre (ou le fichier) que vous recherchez.

Objectif général

- Comprendre en profondeur les systèmes de fichiers, leur implémentation, les technologies RAID associées, et être en mesure de prendre des décisions éclairées sur la gestion et la protection des données dans un environnement informatique.

Objectifs spécifiques 1/2

- Acquérir une compréhension approfondie des concepts fondamentaux des systèmes de fichiers;
- Maîtriser les compétences nécessaires pour implémenter et administrer efficacement des systèmes de fichiers;

- Comprendre les technologies RAID (Redundant Array of Independent Disks) et être capable de choisir et configurer la meilleure solution RAID en fonction des exigences de tolérance aux pannes et de performances du système.

PLAN

INTRODUCTION

I. IMPLÉMENTATION DES SYSTÈMES DE FICHIERS

II. TECHNOLOGIES RAID

CONCLUSION

INTRODUCTION

Un système de fichiers est une méthode de stockage et d'organisation de données sur un support de stockage, tel qu'un disque dur, une clé USB, ou un autre dispositif de stockage. Il s'agit d'une partie essentielle du système d'exploitation d'un ordinateur, chargée de gérer la manière dont les données sont stockées, organisées, nommées, accédées et modifiées.

En termes simples, un système de fichiers permet aux utilisateurs et aux programmes de créer, lire, écrire, supprimer et organiser des fichiers et des répertoires sur un support de stockage. Chaque fichier est identifié par un nom unique au sein du système de fichiers, et il peut contenir des données de différents types, tels que texte, images, programmes, etc.

Le système de fichiers maintient également des informations sur les fichiers, telles que leur emplacement physique sur le disque, leur taille, leur date de création et de modification, ainsi que les autorisations d'accès qui déterminent qui peut lire, écrire ou supprimer un fichier.

En résumé, un système de fichiers est une infrastructure logicielle qui permet la gestion et l'organisation des données stockées sur un support de stockage, offrant une interface permettant aux utilisateurs et aux logiciels d'interagir avec ces données de manière efficace et cohérente.

- ❑ Vos programmes utilisent **le stockage persistant**
 - ❑ On veut garder les données même après avoir éteint l'ordinateur
 - ❑ Potentiellement de très grands volumes de données

- ❑ Solution : **les fichiers** (évidemment)
- ❑ Question: **comment un OS gère-t-il les fichiers ?**
 - ❑ Structure
 - ❑ Nommage
 - ❑ Contrôle d'accès
 - ❑ Protection
 - ❑ Implémentation
 - ❑ etc.

- ❑ Combien de caractères ?
 - ❑ Par exemple : 8+3 ou 255 caractères
- ❑ Sensibles à la casse?
 - ❑ maria vs. Maria vs. MARIA

- ❑ Caractères spéciaux ?
 - ❑ Exemple : 2?,slidês.pdf
 - ❑ Extensions de noms pour indiquer le type de fichier
 - ❑ Exemple : slides.pdf, slides.pdf.gz
- ❑ Est-ce que l'OS doit gérer directement ces extensions de noms de fichier ?

- ❑ Tous les OS supportent les fichiers **normaux** et les **répertoires**
- ❑ Il existe d'autres types de fichiers :
 - ❑ **Fichiers spéciaux "caractères"**
 - ❑ Une abstraction commode pour représenter des périphériques adressables par caractères (souris, clavier. . .)

☐ Fichiers spéciaux "blocs"

- ☐ Une abstraction commode pour représenter des périphériques

- adressables par blocs (disques. . .)

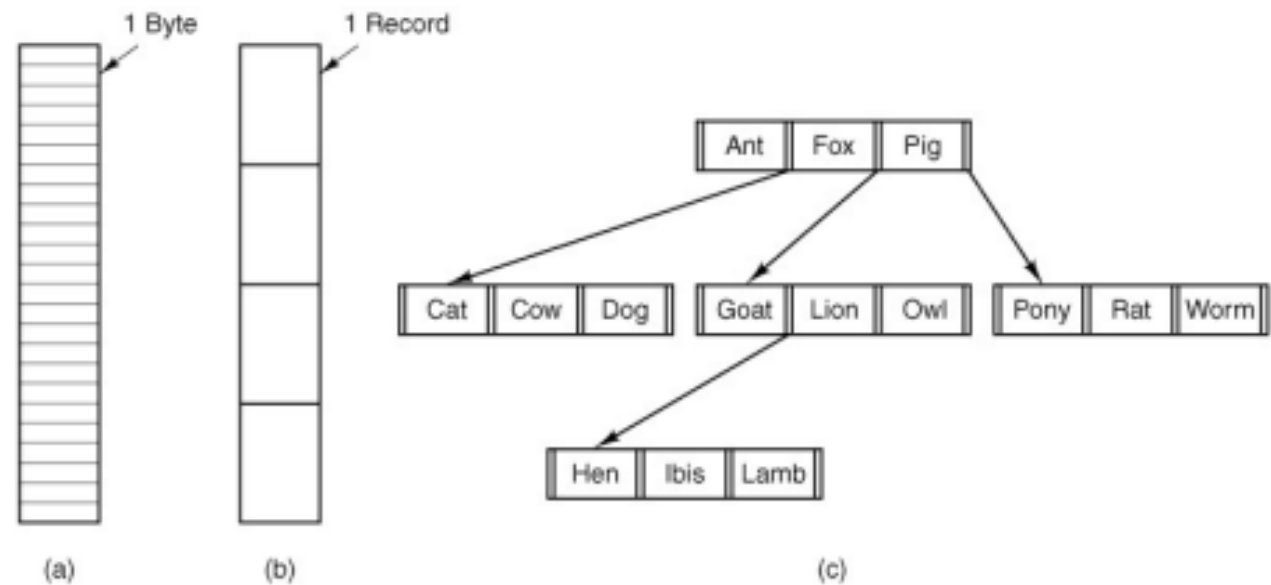
- ☐ Fichiers de métadonnées (Windows)

- ☐ Etc.

- ☐ Fortement typés (MacOS) ou faiblement typés (Linux)

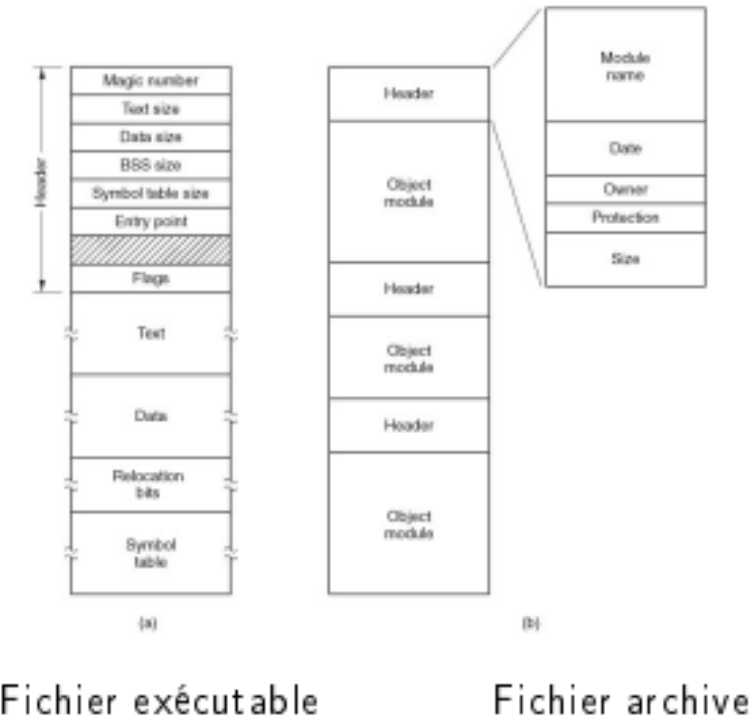
ORGANISATION D'UN FICHER

- ❑ Une séquence d'**octets** ?
- ❑ Une séquence d'**enregistrements** ?
- ❑ **Autre chose** ?



FICHIERS RÉGULIERS == SÉQUENCES D'OCTET

- ❑ Dans les OS modernes : fichier régulier == séquence d'octets
- ❑ Mais ça n'empêche pas de donner une structure interne à la séquence d'octets. . .



ATTRIBUTS D'UN FICHIER

- ❑ On attache des attributs à chaque fichier qui contiennent des méta-données à propos du fichier

Attribute	Meaning
Protection	Who can access the file and in what way
Password	Password needed to access the file
Creator	ID of the person who created the file
Owner	Current owner
Read-only flag	0 for read/write; 1 for read only
Hidden flag	0 for normal; 1 for do not display in listings
System flag	0 for normal files; 1 for system file
Archive flag	0 for has been backed up; 1 for needs to be backed up
ASCII/binary flag	0 for ASCII file; 1 for binary file
Creation time	Date and time the file was created
Time of last access	Date and time the file was last accessed
Time of last change	Date and time the file has last changed
Current size	Number of bytes in the file
Maximum size	Number of bytes the file may grow to

- ❑ Créer un nouveau fichier vide
- ❑ Supprimer un fichier
- ❑ Ouvrir un fichier
 - ❑ L'OS lit les attributs du fichier et détermine sa localisation sur disque avant de donner (ou non) accès au fichier
- ❑ Fermer
 - ❑ L'OS libère les informations qu'il gardait en mémoire à propos du fichier (attributs, position dans le fichier, . . .)

☐ Lire

- ☐ Généralement : lire depuis la position actuelle

☐ Écrire

- ☐ Depuis la position actuelle (en écrasant éventuellement des contenus)

☐ Rajouter

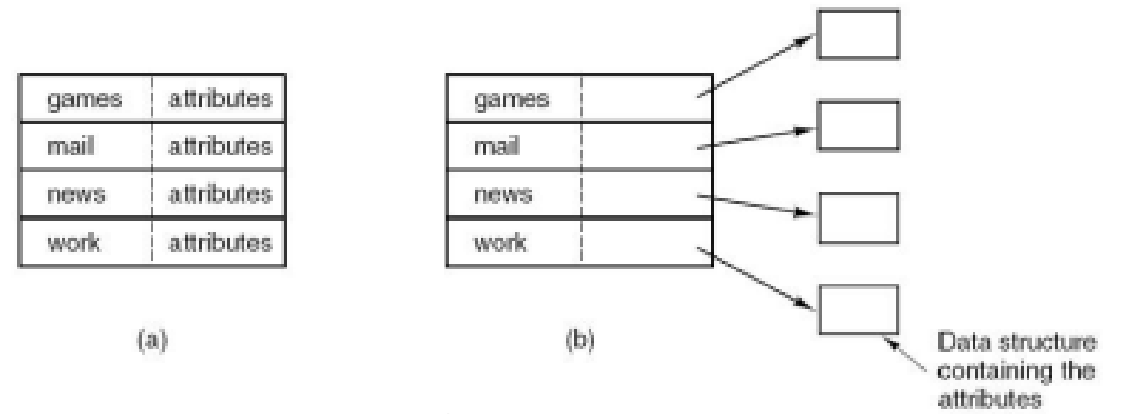
- ☐ Ecrire à la fin du fichier

☐ Déplacer

- ☐ Changer de position dans le fichier

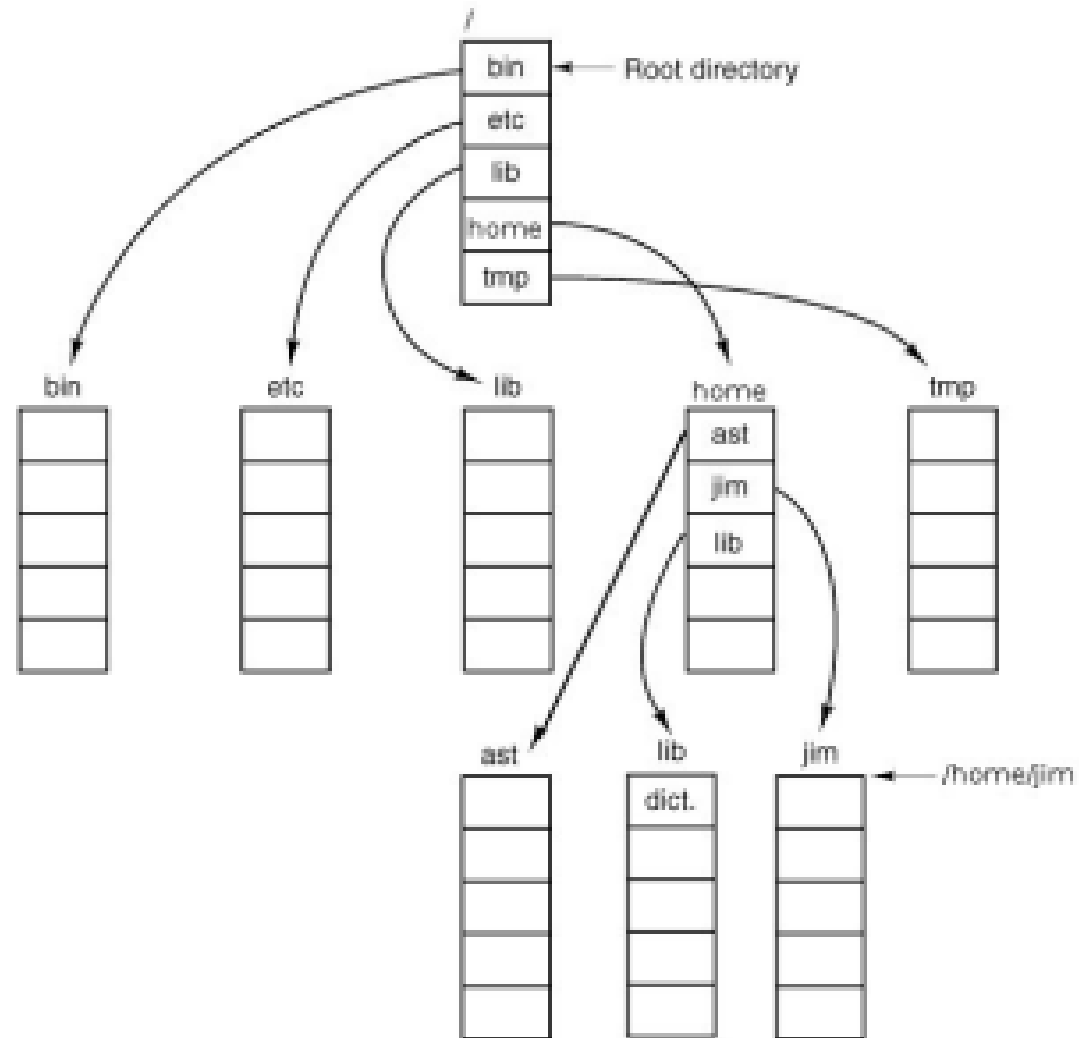
- ☐ Lire les attributs
- ☐ Changer les attributs
- ☐ Renommer le fichier
- ☐ Verrouiller/déverrouiller
 - ☐ Pour s'assurer qu'aucun autre

- ❑ Un répertoire est un fichier (presque) comme les autres
 - ❑ Il contient une liste de fichiers. . .
 - ❑ . . . et leurs attributs



- ❑ Un répertoire peut contenir les noms/attributs d'autres répertoires
 - ❑ Structure hiérarchique
 - ❑ Chemins: `\home\gpierre\mailbox` ou `/home/gpierre/mailbox` ou `../../mailbox`

ARBORESCENCE STANDARD DES SYSTÈMES UNIX



- ❑ Créer un répertoire vide

- ❑ Les entrées spéciales . et .. sont créées automatiquement

- ❑ Supprimer

- ❑ Possible seulement si le répertoire est vide! (on ne veut pas perdre de fichiers)

- ❑ Opendir

- ❑ Closedir

- ☐ Readdir

 - ☐ Lit une entrée du répertoire

- ☐ Renommer

- ☐ Renomme le répertoire

- ☐ Link/unlink

 - ☐ Permet à un même fichier (pas des copies du fichier!)
d'apparaître dans plusieurs répertoires à la fois

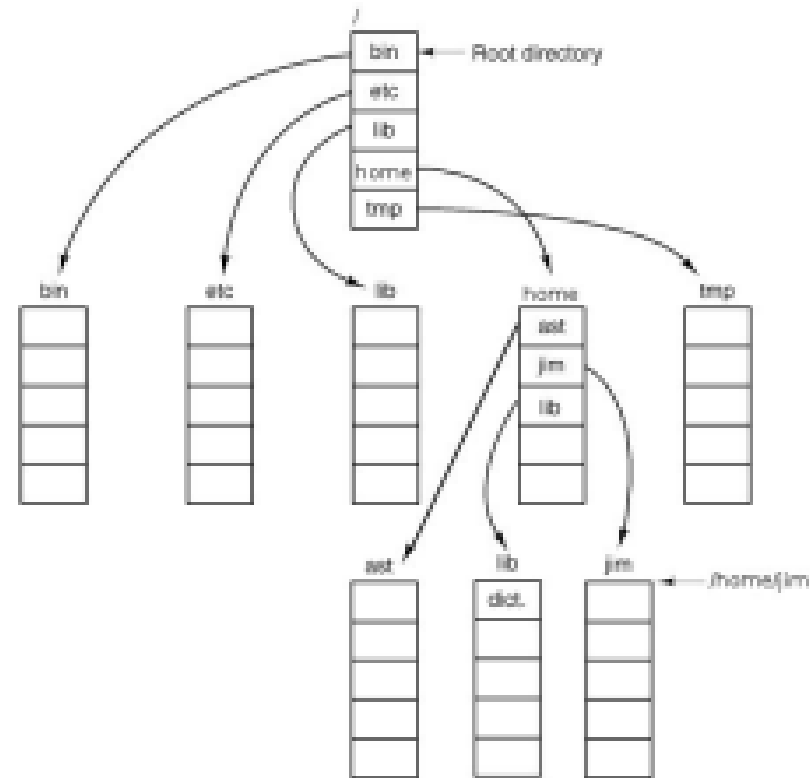
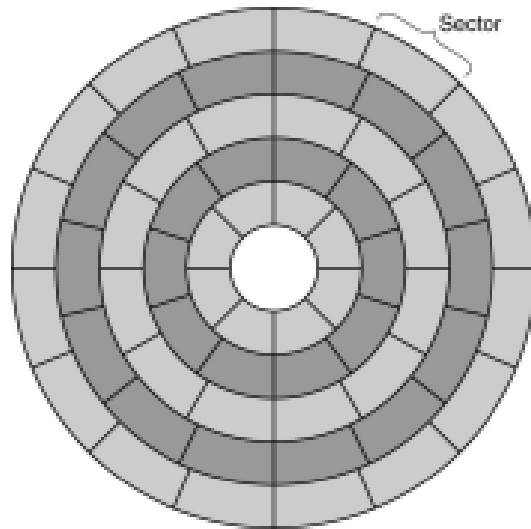
PLAN

INTRODUCTION

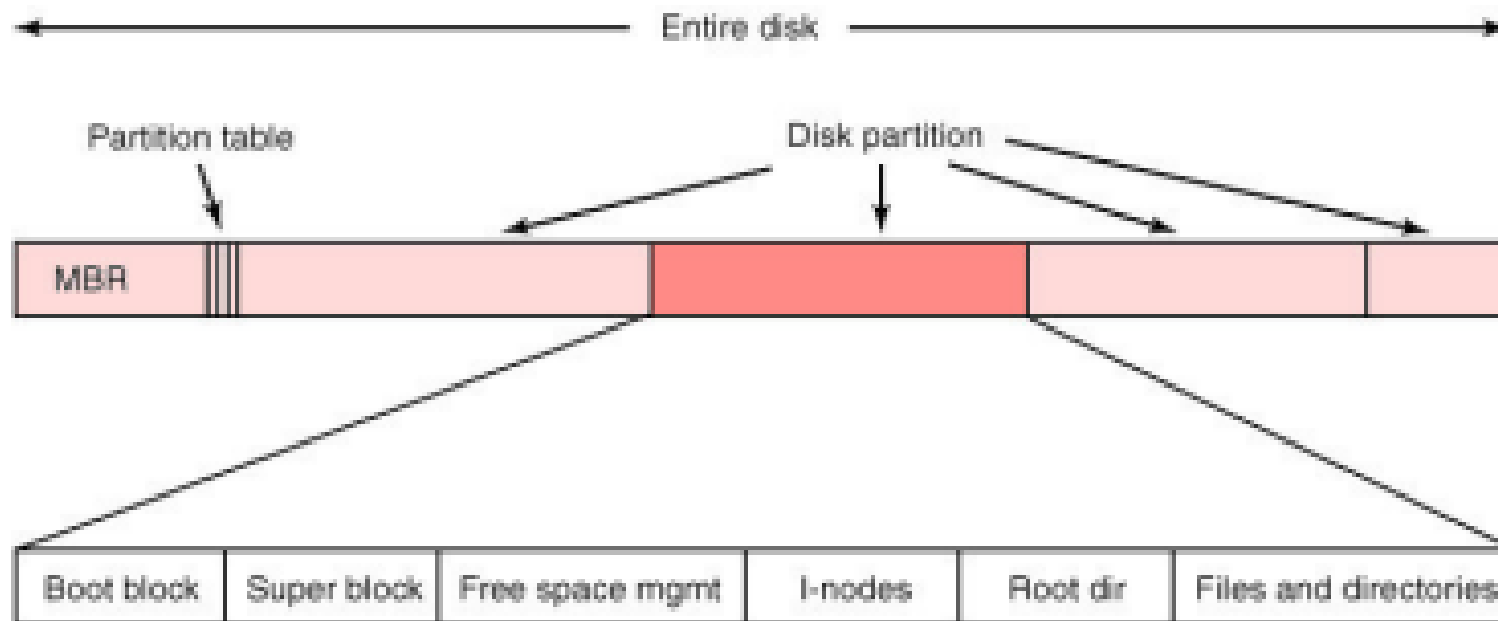
I. Implémentation des systèmes de fichiers

II. Technologies RAID

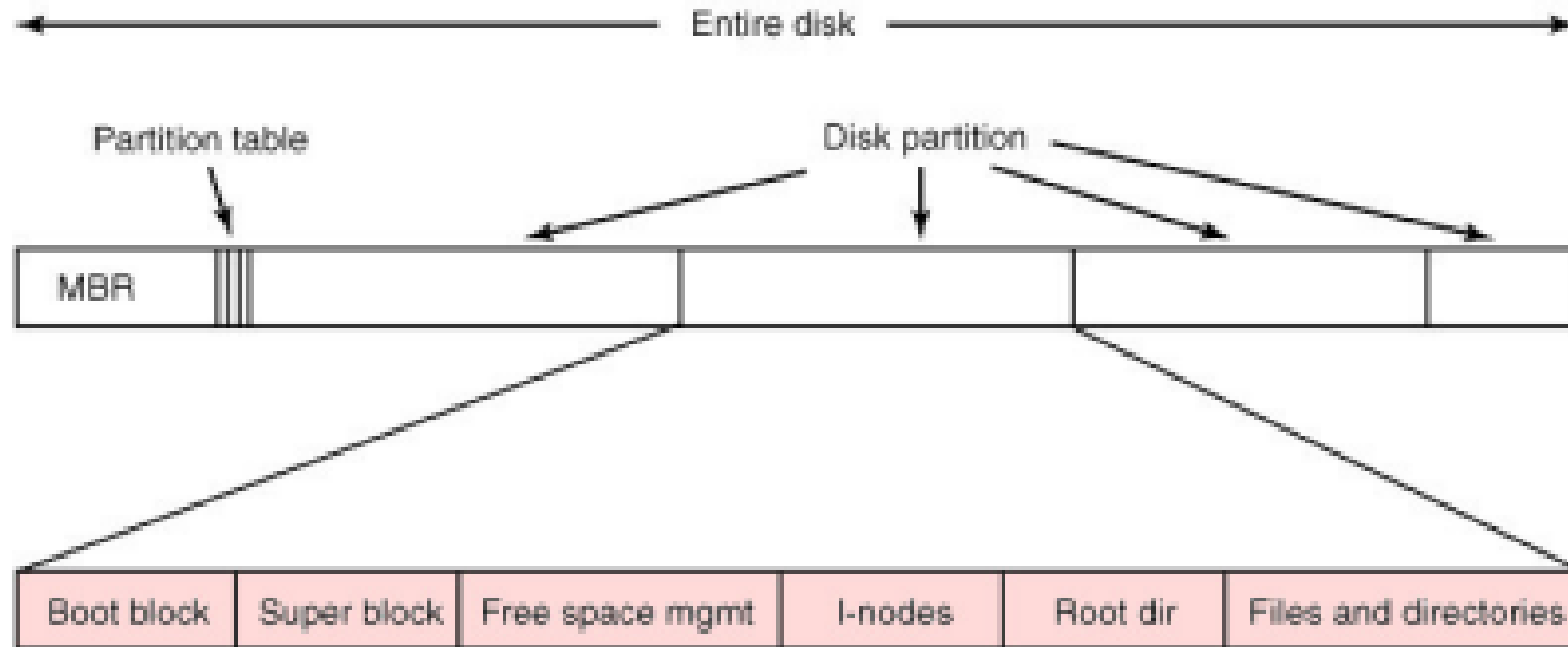
CONCLUSION



- ❑ Les disques sont généralement **partitionnés**
- ❑ **MBR = Master Boot Record**
 - ❑ Utile pour démarrer l'ordinateur
 - ❑ Contient (ou pointe vers) la **table de partition** :
 - ❑ Numéro de bloc de début/fin de la partition
 - ❑ Type de système de fichiers
- ❑ Un système de fichier utilise une seule partition



- ❑ Tous les systèmes de fichiers commencent par un **boot block**
 - ❑ Vide si le système de fichier n'est pas bootable
- ❑ **Tout le reste dépend du système de fichiers**
 - ❑ Par exemple, dans Unix le **superblock** contient les paramètres de configuration du système de fichier
 - ❑ Un même OS peut supporter plusieurs types de systèmes de fichiers :
bfs (BeOS), cramfs (Compressed RAM), ext2, ext3, ext4, minix, msdos, reiserfs, vfat (Windows FAT16, FAT32), iso9660 (CD-ROM) etc.



- ❑ Nous devons garder une trace des **correspondances fichier/blocs**
 - ❑ Quels sont les blocs qui constituent un fichier ? Dans quel ordre ?
- ❑ Une idée très simple : implémentation contigüe
 - ❑ Chaque fichier commence à partir d'un certain numéro de bloc
 - ❑ Chaque fichier utilise **n blocs contigus**
 - ❑ **Bien** : accès très efficace au fichier
 - ❑ **Bien** : il est facile de se déplacer dans le fichier

❑ Très très mauvais :

- ❑ Les fichiers ne peuvent pas grandir
- ❑ Si les fichiers se rétrécissent (ou sont supprimés) cela crée des trous entre les fichiers

❑ Question : dans quelles conditions cette stratégie est-elle utile ?

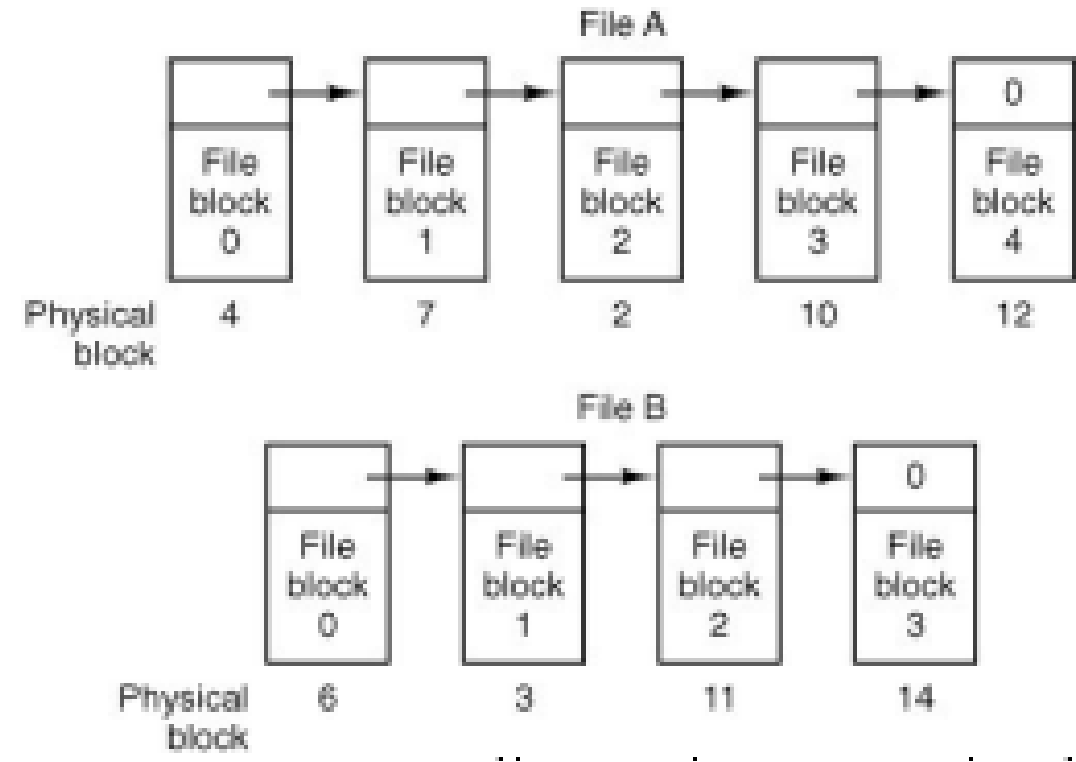
Systèmes de fichier en lecture seule

(par ex: CD-ROM)

❑ Idée : faire démarrer chaque bloc de données par l'identifiant du **prochain bloc** dans le fichier

❑ Le dernier bloc du fichier contient un pointeur nul

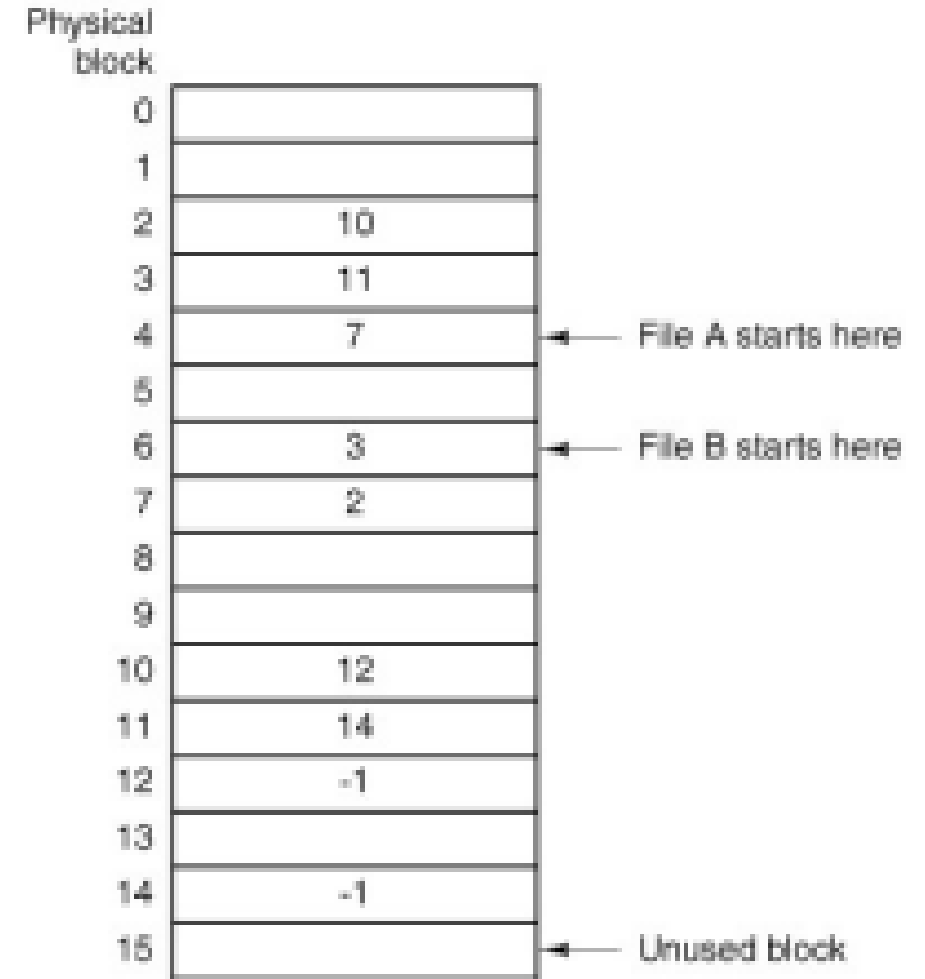
❑ Le reste du bloc contient les données du fichier



- ❑ **Bien** : les fichiers peuvent être redimensionnés à la demande
- ❑ **Bien** : l'accès séquentiel au fichier est assez efficace
- ❑ **Mauvais** : déplacements coûteux dans le fichier
- ❑ **Bizarre** : les données sont stockées par unités bizarre (ex: 508 octets)
 - ❑ Les programmes lisent/écrivent souvent les données par puissance de 2 (par exemple: 1024 octets)

AMÉLIORATIONS DE LA STRUCTURE PAR LISTE CHAÎNÉE 1/2

- ❑ Rajoutons des méta-données : une **File Allocation Table (FAT)**
 - ❑ Une seule FAT pour l'ensemble du système de fichiers
 - ❑ La FAT sépare la liste chaînée (méta-données) des données du fichier

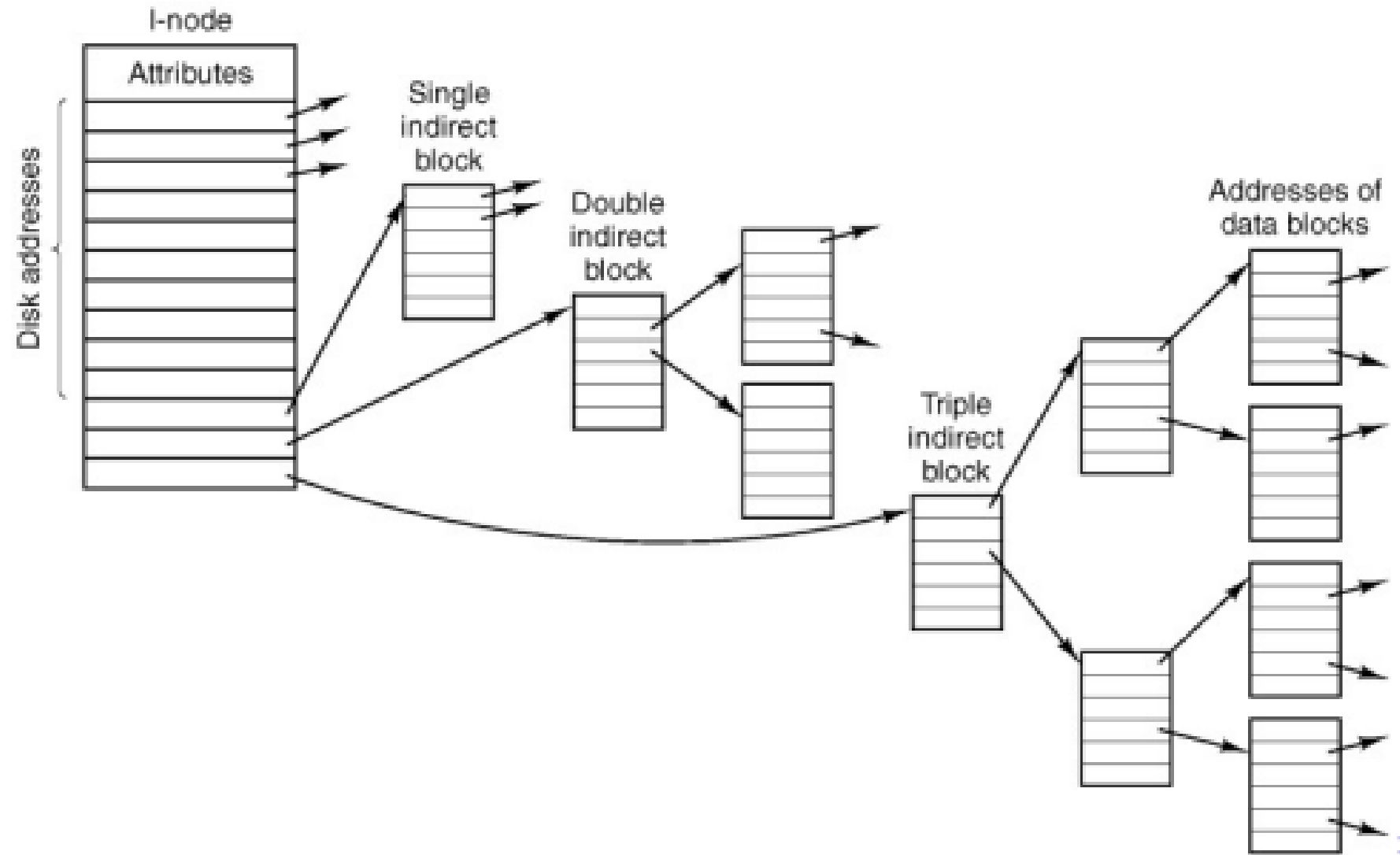


- ❑ **Bien** : Les déplacements dans le fichier sont plus rapides

- ❑ **Pas très bien** : il faut stocker la FAT en mémoire 500 GB = 500 millions de blocs de 1-KB

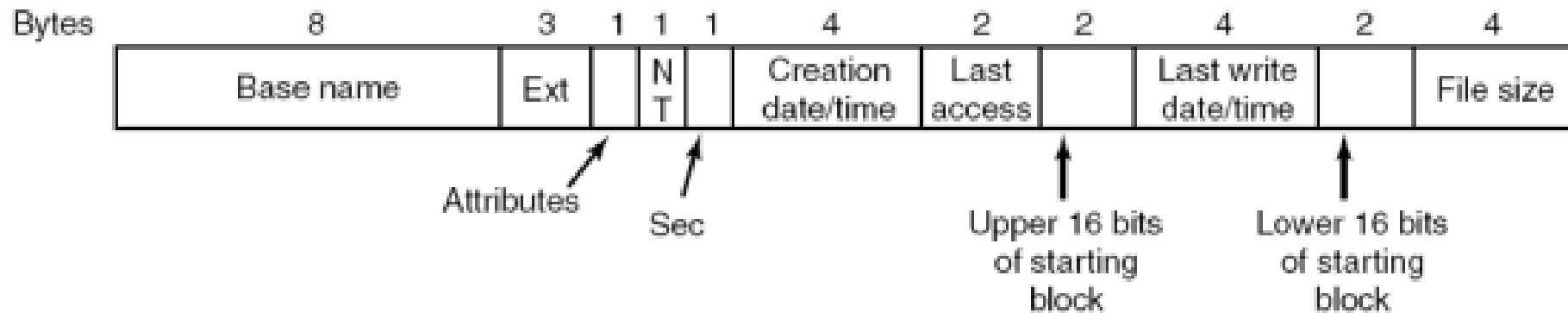
Les systèmes de fichiers Windows marchent comme ça (FAT16, FAT32)...

- ❑ Un **inode** (index-node) est un bloc qui contient les méta-informations d'un fichier, y compris la liste de ses blocs de données
 - ❑ Un seul inode par fichier
 - ❑ Un inode peut stocker n adresses de blocs de données



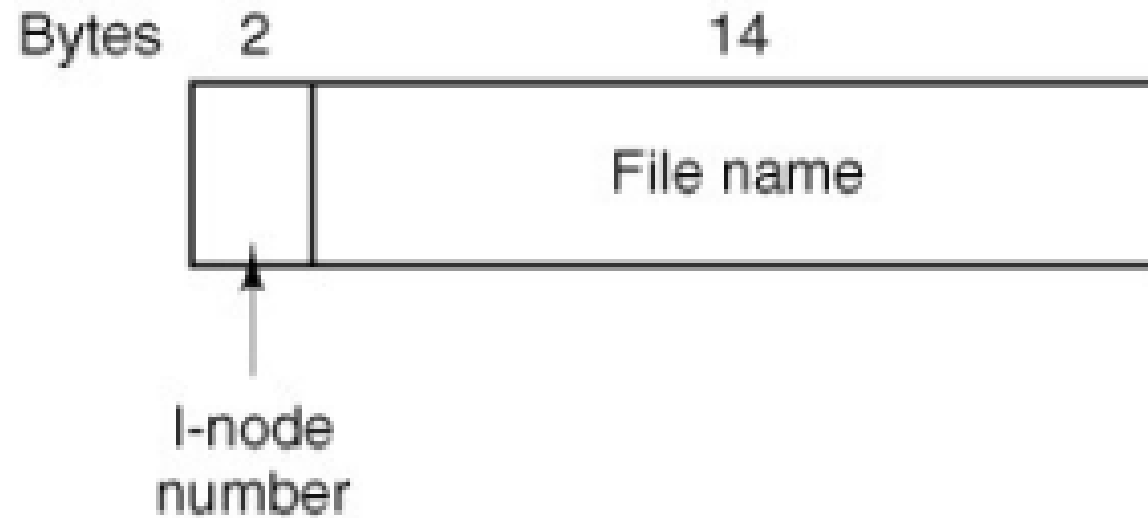
IMPLÉMENTATION DES RÉPERTOIRES DANS WINDOWS

- ❑ Les vieux systèmes de fichiers Windows supportent des noms de fichiers sur 8 caractères + 3 caractères d'extension (par ex: foobarba.pdf)



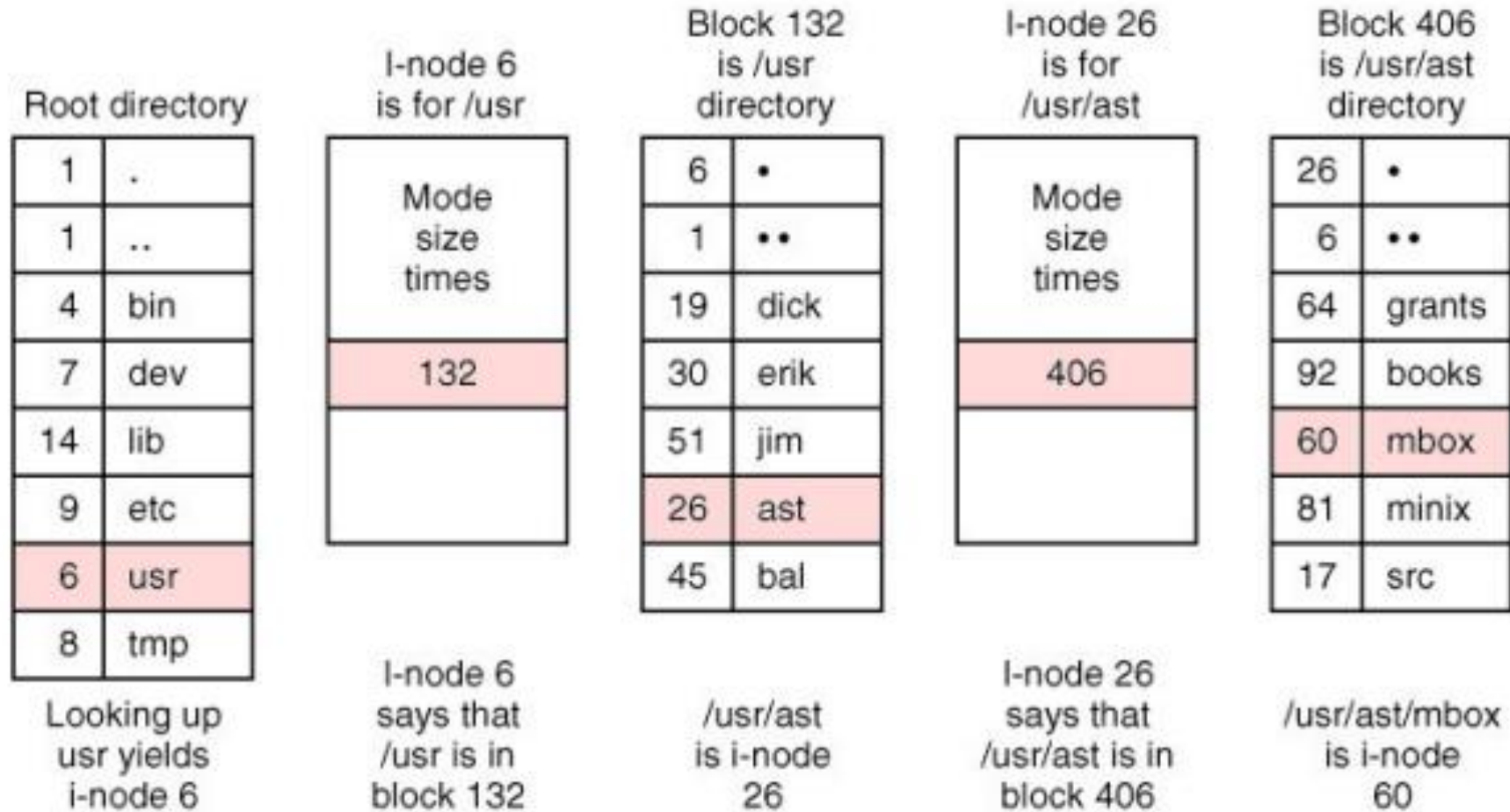
(Il y a une astuce pour permettre des noms de fichiers plus longs. . .

- ❑ Les entrées du répertoire sont très simples :

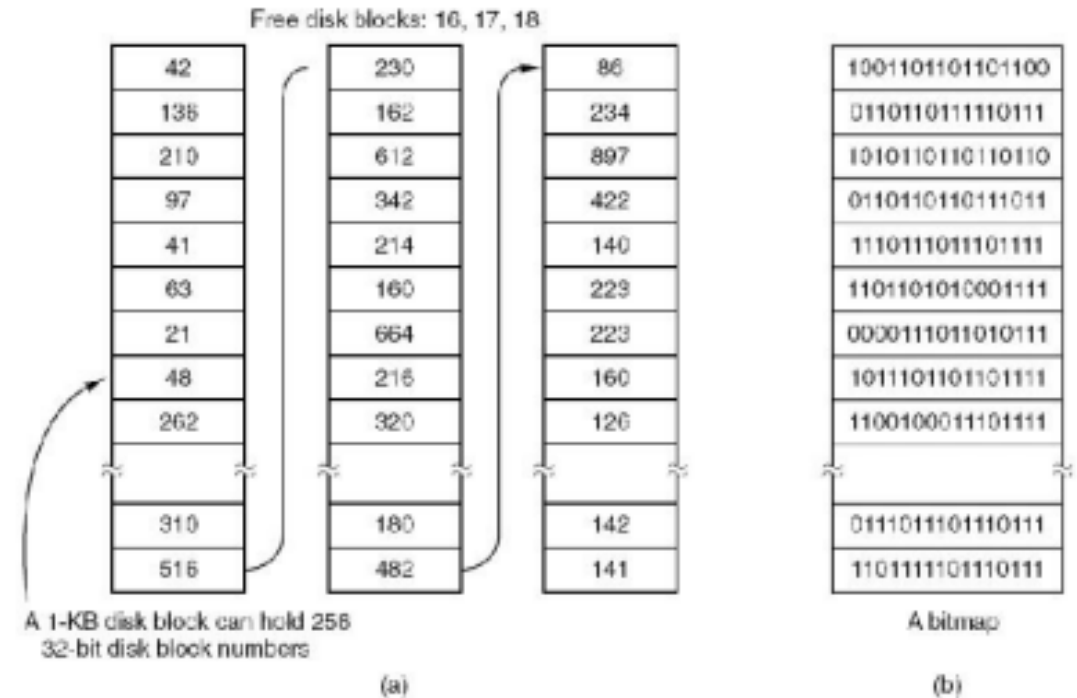


- ❑ Toutes les méta-informations au sujet du fichier sont stockées dans l'inode
 - ❑ Longueur du fichier
 - ❑ propriétaire Mode (type de fichier, droits d'accès)
 - ❑ Timestamps

ÉTAPES POUR TROUVER LE FICHIER /USR/AST/MBOX

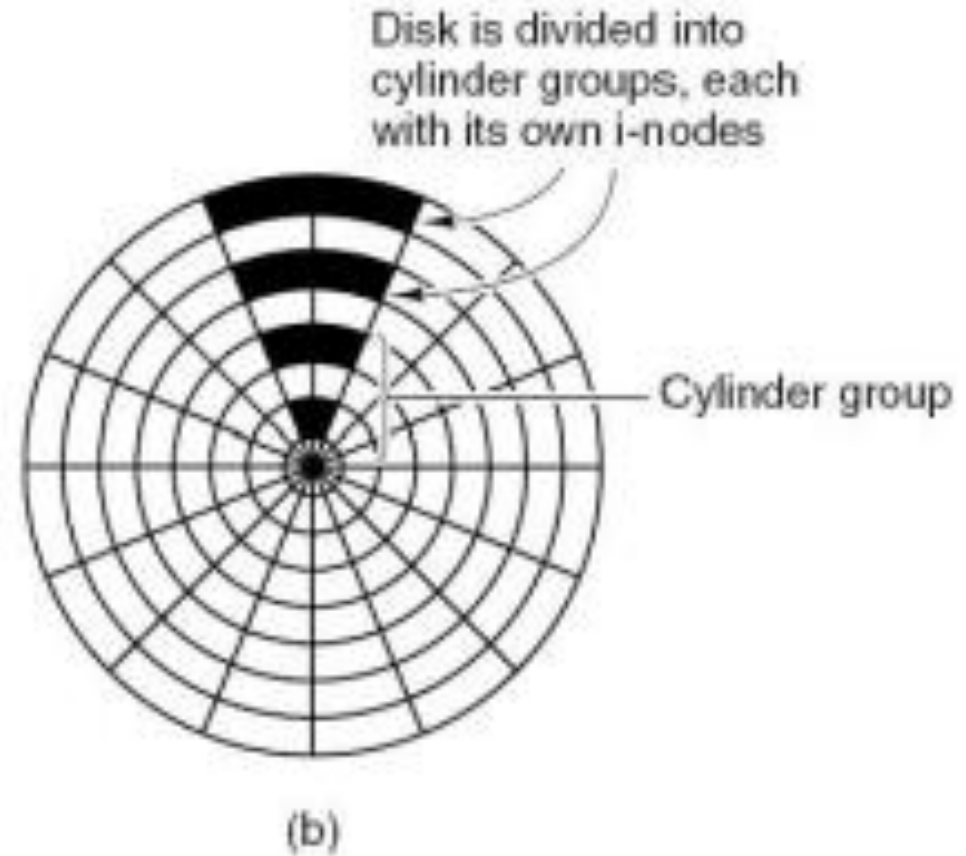
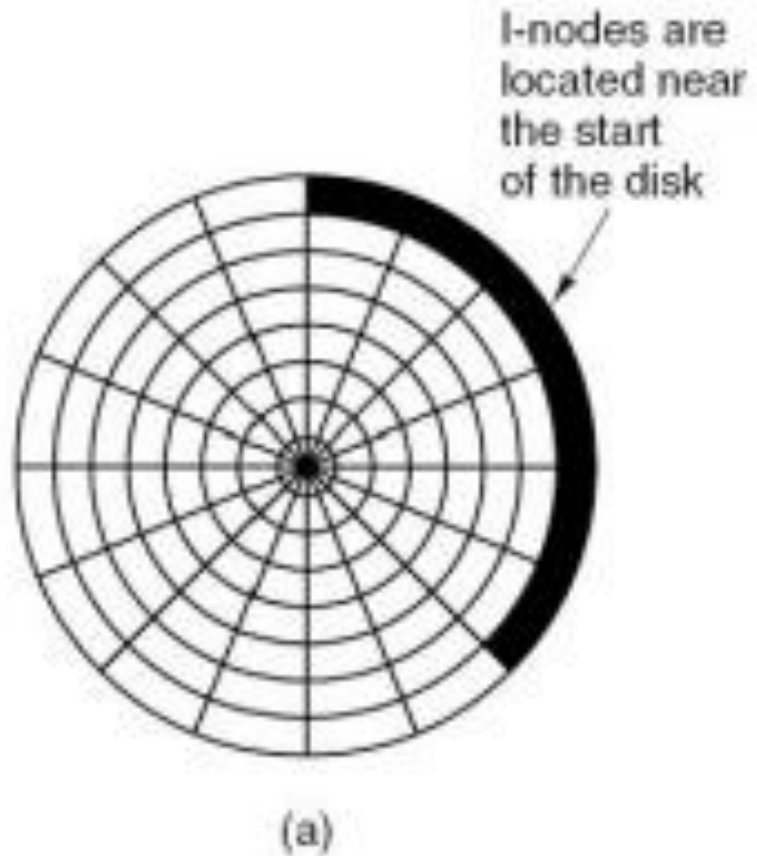


- ❑ On peut maintenir la liste des blocs libres grâce à une liste chaînée ou des bitmaps



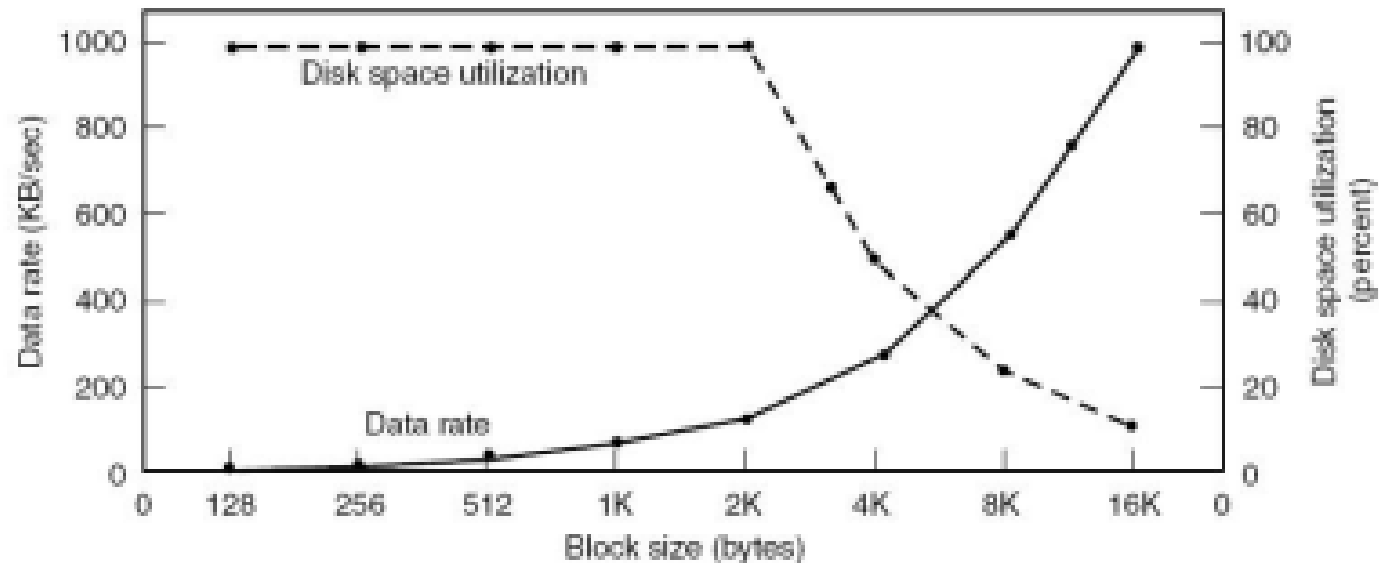
- ❑ Bonne nouvelle : s'il y a beaucoup de blocs libres alors on peut en utiliser certains pour stocker la liste des blocs libres

- ❑ Quand on écrit dans les fichiers il faut **choisir des blocs libres** pour stocker les données
 - ❑ Peut-on choisir intelligemment ?
- ❑ Vieux systèmes de fichiers :
 - ❑ Garde tous les inodes au début du disque
 - ❑ Le reste du disque contient les blocs de données
 - ❑ Mais n'importe quel accès disque demande d'**accéder à l'inode et aux blocs de données!**



- ❑ On n'est pas obligé d'utiliser 1 bloc du système de fichiers = 1 secteur disque
 - ❑ On pourrait par exemple utiliser 1 bloc = 7 secteurs $\Rightarrow$ 1 bloc = 3.5 kB
- ❑ Grandes tailles de blocs :
 - ❑ Bonne performance d'accès (on utilise beaucoup de blocs contigus)
 - ❑ Gaspillage d'espace disque sur le dernier bloc

- ❑ Petites tailles de blocs :
 - ❑ Moins bonne performance
 - ❑ Mais meilleure utilisation de la capacité de stockage
- ❑ Si on suppose que tous les fichiers font exactement 2 kB



PLAN

INTRODUCTION

I. IMPLÉMENTATION DES SYSTÈMES DE FICHIERS

II. TECHNOLOGIES RAID

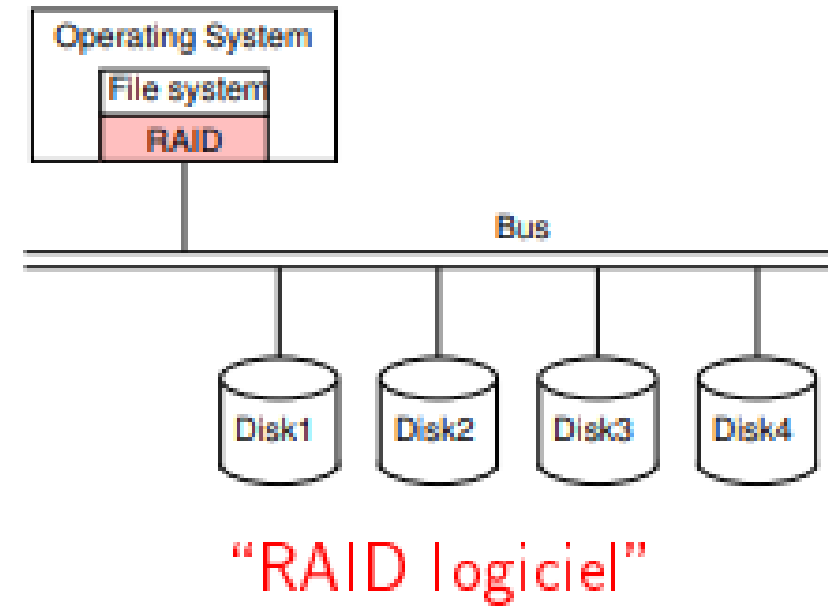
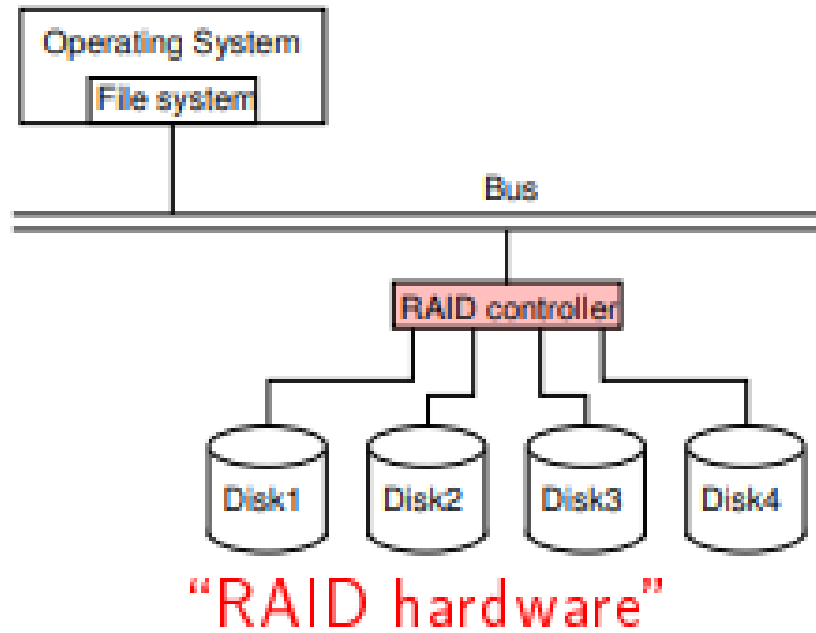
CONCLUSION

❑ RAID = Redundant Array of Inexpensive/Independent
Disks

❑ On utilise plusieurs disques ensemble

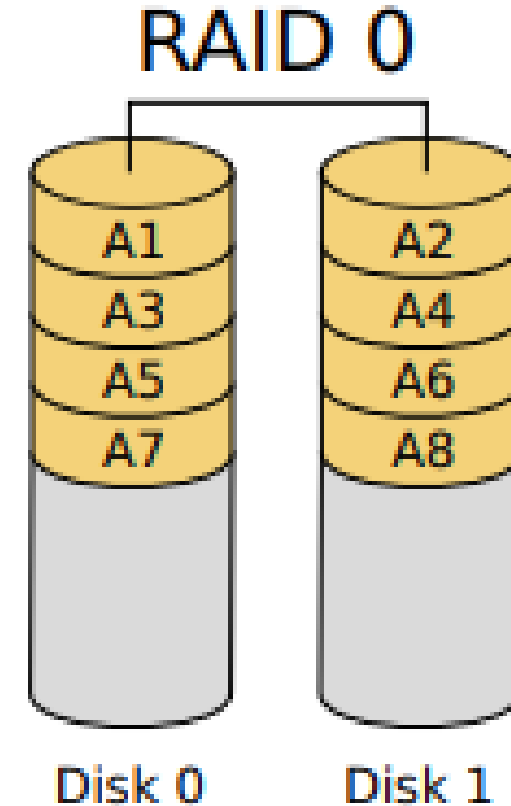
❑ Et on les traite comme une seule entité

❑ RAID peut être implémenté dans le contrôleur des disques
(RAID hardware) ou dans le système d'exploitation (RAID
logiciel)

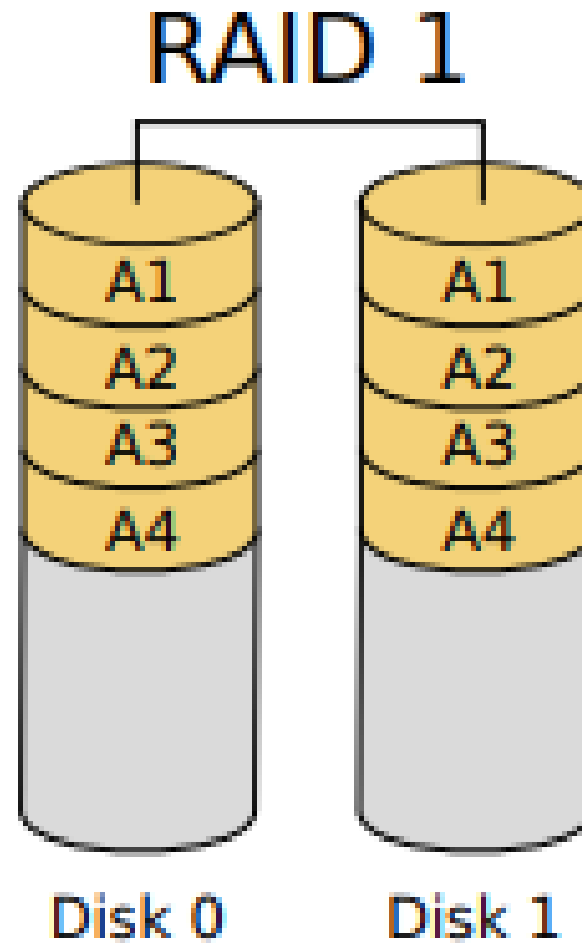


RAID 0 : STRPING

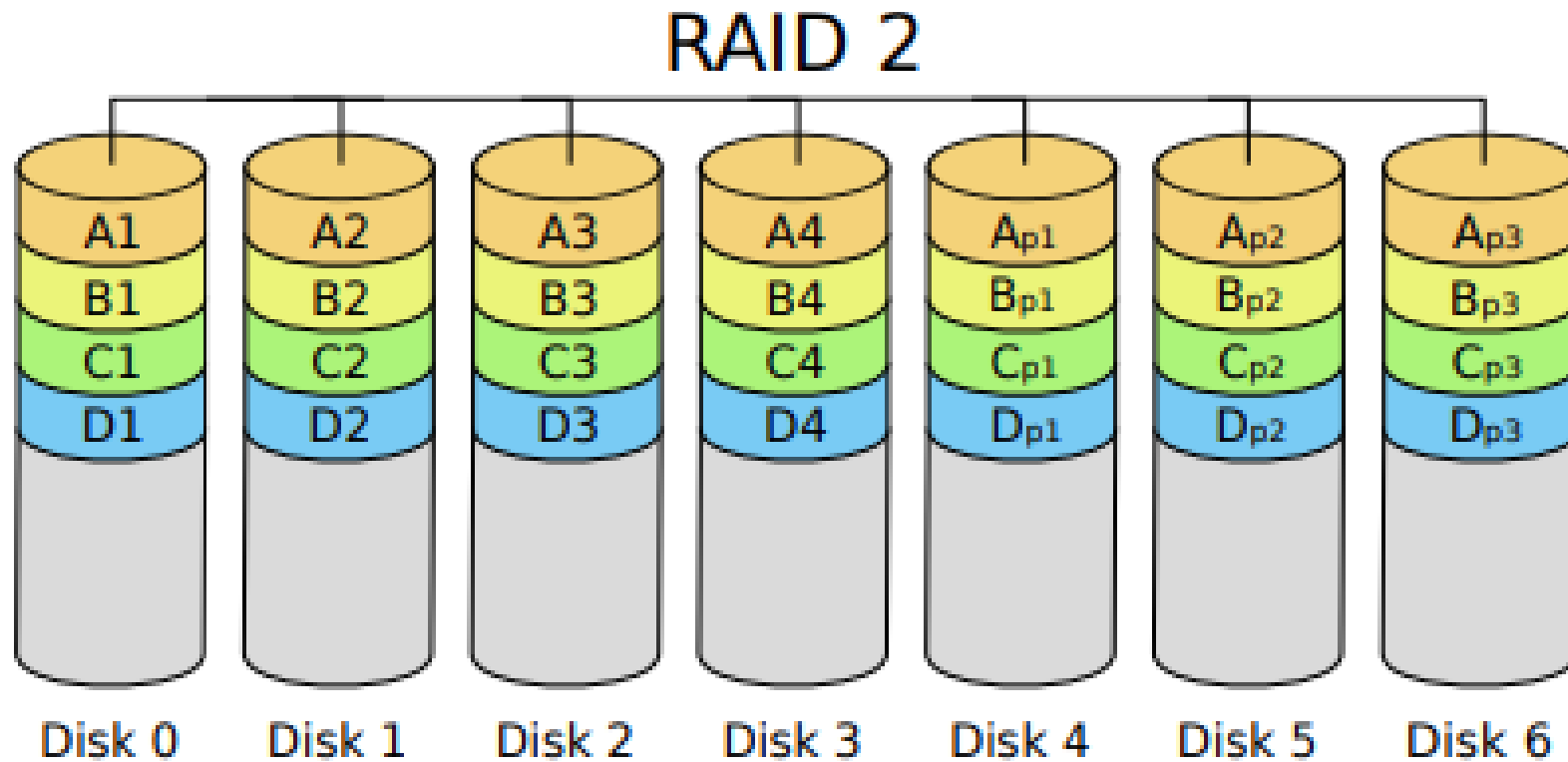
- ❑ **Striping** = distribuer les secteurs entre plusieurs disques
 - ❑ Si le programme accède à plusieurs secteurs consécutifs on peut faire les accès en parallèle
⇒ meilleure performance
- ❑ Pas de redondance ⇒ pas de gain de fiabilité



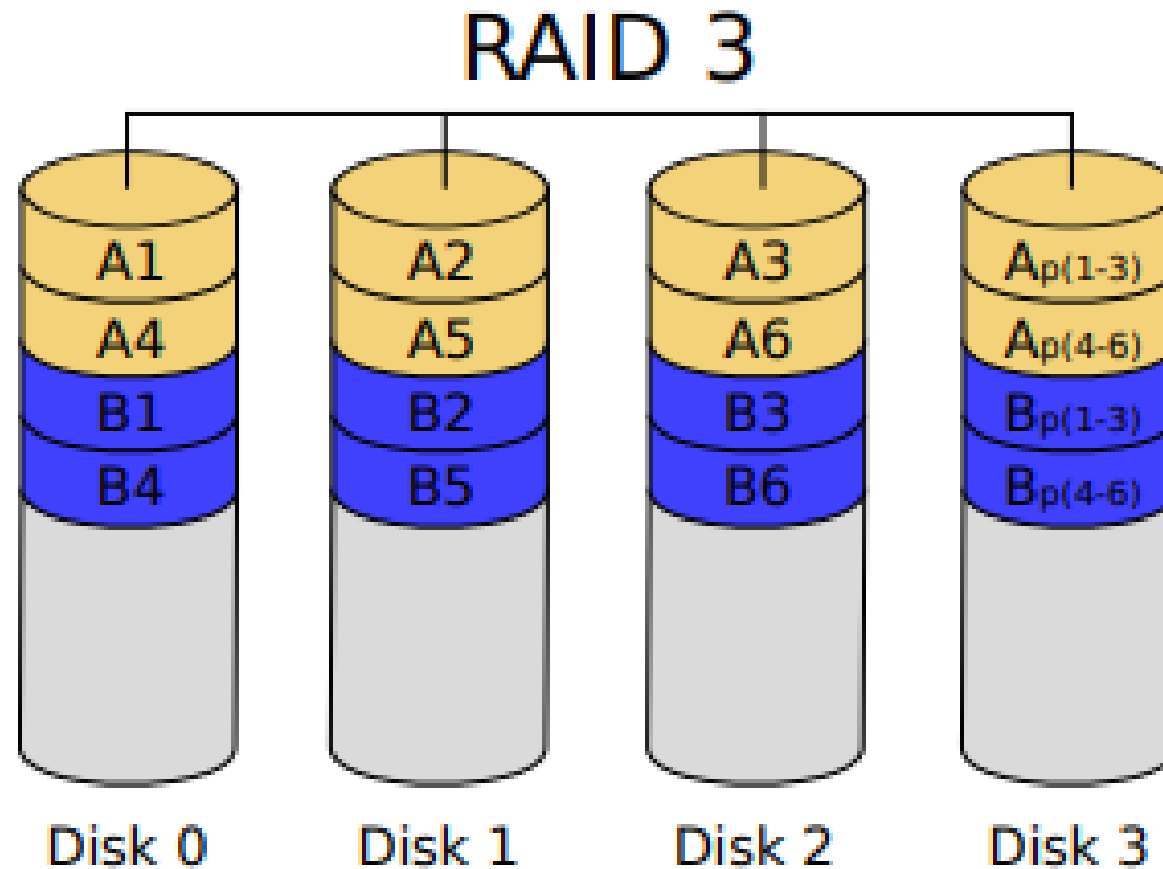
- ❑ **Réplication** = création de plusieurs copies identiques sur des disques différents
 - ❑ On peut faire les lectures depuis n'importe quelle copie $\Rightarrow$ bonnes performances en lecture
 - ❑ Il faut faire les écritures sur tous les disques $\Rightarrow$ pas de gain de performance en écriture
 - ❑ Si un disque se plante on peut accéder aux données sur les autres disques $\Rightarrow$ bonne fiabilité
 - ❑ On utilise **N fois plus d'espace disque!** $\Rightarrow$ assez cher



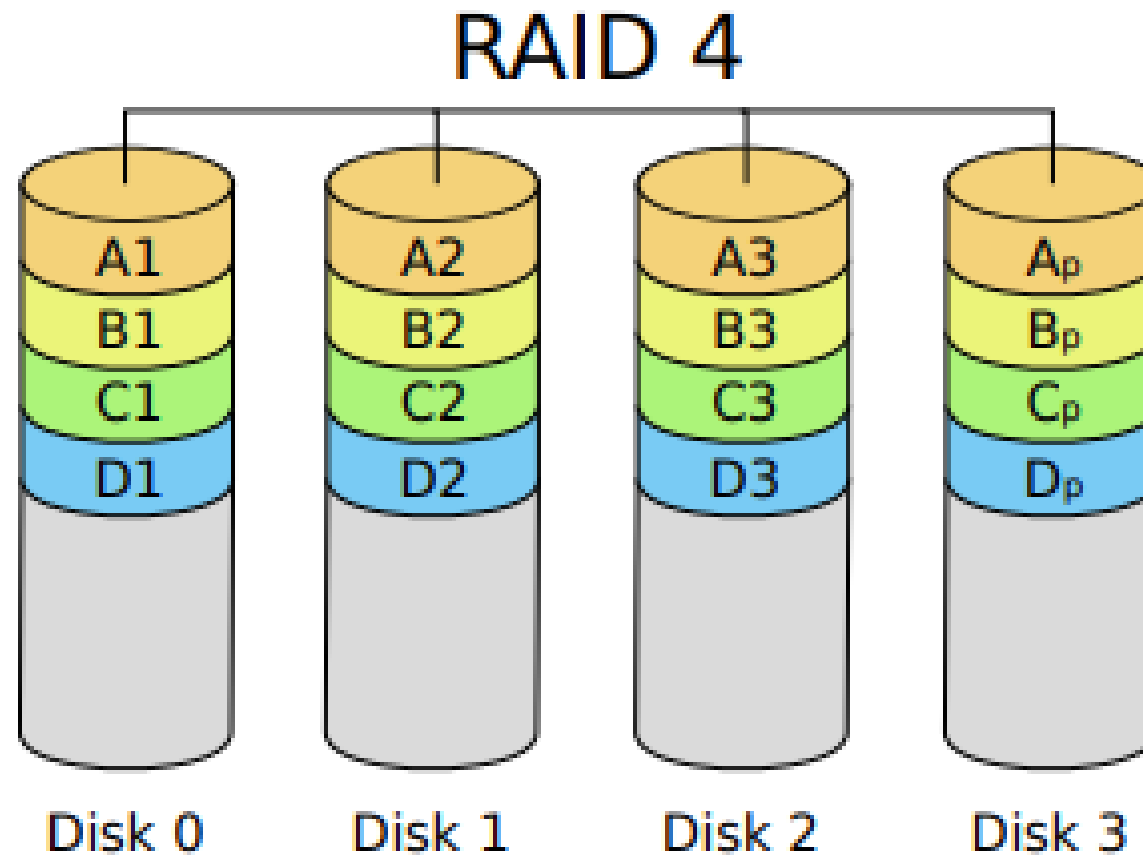
- ❑ Un code de Hamming permet de **détecter** et de **corriger** des erreurs, mais avec moins de redondance que la réplication
 - ❑ **Les bits de chaque octets sont distribués** sur plusieurs disques
 - ❑ Et on stocke le code de Hamming sur des disques à part
 - ❑ Cela permet de détecter et corriger des erreurs d'un bit



- ❑ Plus simple que le code de Hamming: on utilise des bits de parité
- ❑ Cela permet de **détecter les erreurs** (mais pas de les réparer)
- ❑ En cas de plantage du disque, on sait où est l'erreur, on peut la réparer $\Rightarrow$ bonne fiabilité



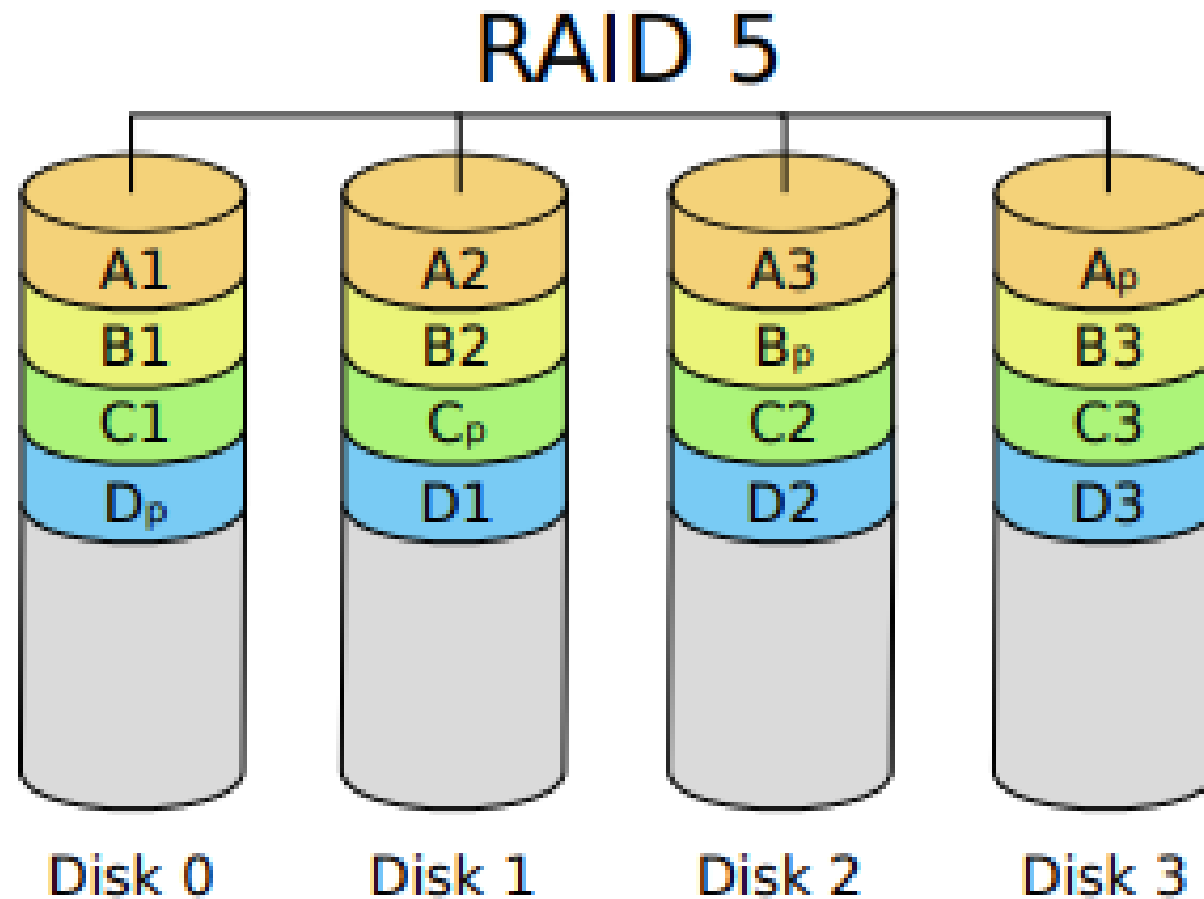
- ❑ Similaire à RAID3, mais il fonctionne **bloc par bloc**
- ❑ Mêmes bonnes propriétés que RAID3
- ❑ Une nouvelle bonne propriété : on peut de nouveau accélérer les lectures/écritures
- ❑ Mais : pour chaque petite écriture il faut lire tous les blocs pour régénérer les blocs de parité



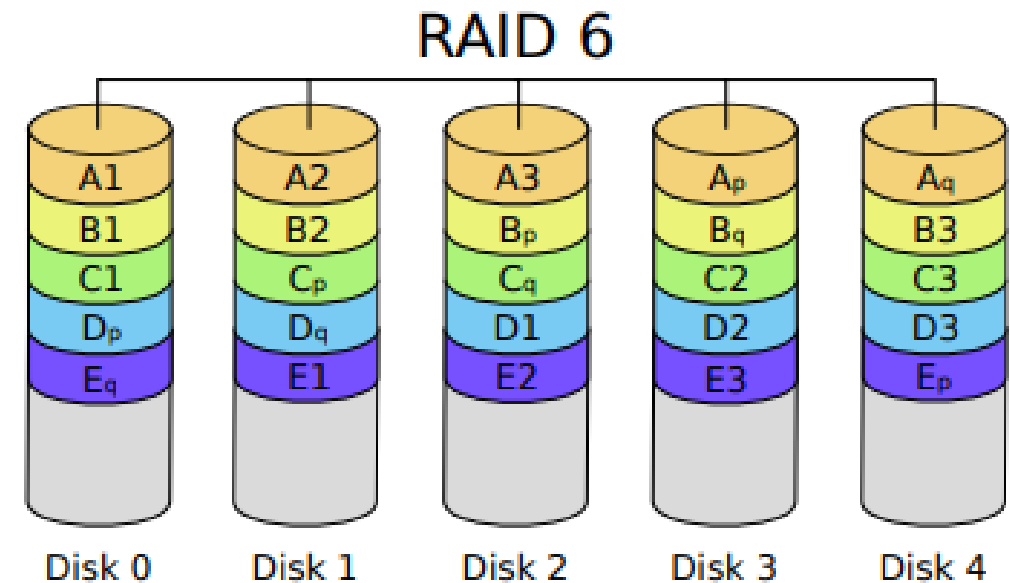
RAID 5 : DISTRIBUTION SYMÉTRIQUE DES BLOCS DE PARITÉ 1/2

- ❑ Similaire à RAID4, mais les blocs de données et de parité sont stockés sur les mêmes disques
- ❑ Cela répartit la charge de calculer les bits de parité (au moins avec du RAID hardware)

RAID 5 : DISTRIBUTION SYMÉTRIQUE DES BLOCS DE PARITÉ 2/2



- ❑ Similaire à RAID 5 mais avec **deux fois plus de blocs de parité**
- ❑ On peut supporter la perte de **deux disques à la fois**



Les configurations RAID les plus utilisées sont RAID 1 (réplication totale), RAID 5 (avec de la redondance), RAID 6 (avec une double dose de redondance).

Bien que l'implémentation des systèmes de fichiers et l'utilisation des technologies RAID puissent sembler complexes à première vue, il est clair que ces éléments jouent un rôle essentiel dans la gestion de données et la préservation de l'intégrité des informations critiques. L'analyse de ces concepts nous permet de comprendre leur importance dans le domaine de la gestion des données et de la sauvegarde des informations.

En conclusion, l'adoption judicieuse de systèmes de fichiers appropriés et de configurations RAID adaptées à chaque contexte est essentielle pour garantir la disponibilité, la redondance et la résilience des données, assurant ainsi une meilleure gestion et protection des informations dans un monde de plus en plus numérique et connecté. En continuant à suivre les meilleures pratiques et à surveiller les évolutions technologiques, les organisations peuvent tirer parti de ces outils pour améliorer leur efficacité opérationnelle et renforcer leur sécurité informatique.